Università degli Studi di Trento Corso di Programmazione I – E3: 145430/145935 4° Appello – 08/07/2022 – a.a. 2021/2022	**PC N°**
COGNOME ..... **NOME .....** **MATRICOLA**	
RITIRATO	**FIRMA PER RITIRARSI .....**

NON è consentito utilizzo di alcun strumento elettronico, informatico etc. durante la prova.
Fa parte della valutazione la leggibilità *del codice C/C++.*
Tempo a disposizione: 100 minuti.

PARTE TEORICA: 12 PUNTI
Tempo a disposizione consigliato: 30 minuti.

[Domanda1 – 2 punti]
Quanto vale la somma tra $F9_{16}$ e $2A2_{16}$ in base 8?
Suggerimento: convertire i numeri esadecimali in binario, sommarli e convertirle il risultato in ottale (mostrare il procedimento con tutti i passaggi in dettaglio!)

[Domanda2 – 2 punti]
Una variabile di tipo FILE (per esempio FILE* fp;) è una struttura che contiene cosa? Dettagliare gli elementi di cui è costituita!

[Domanda3 – 2 punti] rispondere SI o NO alle seguenti affermazioni:

SI ☐ NO ☐ L'istruzione break permette di interrompere l'esecuzione di un metodo.
SI ☐ NO ☐ L'unico uso possibile dell'istruzione break è per interrompere l'esecuzione di un ciclo.
SI ☐ NO ☐ Il break può essere usato nel ramo if di una istruzione if..else per saltare al ramo else.
SI ☐ NO ☐ Il break può essere usato per interrompere un ciclo, ma si può scrivere cicli senza usarlo.
SI ☐ NO ☐ Il corpo del ciclo while può essere eseguito indefinitamente.
SI ☐ NO ☐ Il corpo del ciclo while può non essere mai eseguito, se la condizione del ciclo risulta falsa.
SI ☐ NO ☐ La condizione del ciclo while viene valutata prima dell'eventuale esecuzione del corpo del ciclo.
SI ☐ NO ☐ La condizione del ciclo while viene valutata solo una volta.
SI ☐ NO ☐ Il ciclo do-while consente di esprimere solo cicli indefiniti.
SI ☐ NO ☐ Il ciclo do-while termina sempre.
SI ☐ NO ☐ Il ciclo do-while non si può usare per fare l'input di una sequenza di dati.
SI ☐ NO ☐ Il corpo di un ciclo do-while potrebbe non essere mai eseguito.

[Domanda4 - 4punti]

Risolvere la seguente equazione di ricorrenza, specificando il limite asintotico superiore (si usi il metodo iterativo).

$$\begin{cases} T(1) = N \\ T(N) = T(N/2) + 2 \end{cases}$$

[Domanda 5 - punti 2]

In base alle espressioni riportate dire cosa contengono le variabili x e y:

```
int x = 3; int y= 3;  x = y + (y *= 2);          x [     ]  y [     ]

int x = 1; int y= 2;  x = (y++) + (x = y);       x [     ]  y [     ]

int x = 4; int y= 5;  x = (y++)-(--y);           x [     ]  y [     ]

int x = 1; int y= 2;  x = (y = ++x);             x [     ]  y [     ]

int x = 4; int y= 0;  x = (x++)+(--y);           x [     ]  y [     ]
```

PARTE PRATICA: 21 PUNTI
Tempo a disposizione consigliato: 70 minuti.

[A - punti 6] Inserire i seguenti tipi in un file ***dati.h*** ed implementarlo in un file ***dati.cpp***

```
#define DIM 3
typedef struct Tfattura{
  char richiedente[20];
  float importo;
  int numero;
  Tfattura()    { /*  da implementare */ }
  void stampa() { /*  da implementare → formato: vedi punto D */ }
} Tfattura;
typedef struct Tstack {
   Tfattura* is;   // array con allocazione dinamica
   int dim;        // dimensione stack
   int n;          // num elementi presenti
  /* implementare costruttori 0 param, 1 param (dimensione stack) */
  /* implementare i metodi pop, push, isFull, isEmpty, stampa */
} Tstack; // STACK realizzato con array
```

[B - punti 7] Modificare il file ***main.cpp*** includendo il file ***dati.h*** ed inserendo il seguente codice nel ***main***:

```
TipoStack* sFatt = new TStack[DIM];
/* inizializzare tutti i 3 stack tutti con dim=5 */
/* inserire 10 elementi di tipo Tfattura (usare la funzione newFattura
  per avere un dato inizializzato) nei 3 stack secondo la logica:
  se importo < 10 salvare Fattura su file (funzione salvaFattura)
  se importo < 100 primo stack; se importo > 500 ultimo stack;
  altrimenti secondo stack
  NOTA per utilizzo funzione newFattura: numeroFattura deve essere
  progressivo ed univoco – vedi punto C
*/
stampaStack(sFatt);
/* deallocare tutti gli elementi creati nella memoria Heap */
```

[C - punti 3] Implementare una funzione **`newFattura`** che riceve in input l'indirizzo di una variabile di tipo **`Tfattura`** ed un valore **`int numeroFattura`** e la inizializza con:

- campo **`importo`** valore casuale tra 1.00 e 999.00;
- campo **`numero`** inizializzato con valore passato **`numeroFattura`**.
- campo **`richiedente`** letto da tastiera.

[D - punti 2] Implementare una funzione **`stampaStack`** che riceve in input una variabile di tipo **`TStack * sf`**, e stampa il contenuto dei diversi stack utilizzando opportunamente il metodo **stampa** di **`TStack`**. Nella forma:

numFattura=numero richiedente **importo=**importo

[E - punti 3] Implementare una funzione **`salvaFattura`** che riceve come input una variabile di tipo **`Tfattura f`** e la inserisce in un file denominato "**`FattureKO.txt`**" nella forma:

numFattura=numero richiedente **importo=**importo